AI Assisted Coding

LAB ASSIGNMENT – 15

Backend API Development: Creating RESTful Services with AI

Name : K. SAI VARDHAN REDDY

ROLLNO: 2503A51266

Objective

To build a RESTful backend using FastAPI, implement Create, Read, Update, and Delete (CRUD) operations for products, and use FastAPI's automatic Swagger documentation. The code is presented as editor-style screenshots, followed by terminal/output screenshots, as requested.

Software Requirements

- Python 3.10 or later
- Visual Studio Code
- FastAPI and Uvicorn
- A terminal and browser for testing endpoints

Setup Instructions

Open a terminal in the project folder and install the required packages:

pip install fastapi uvicorn

Create a file named main.py. Add the code shown in the screenshots below. Start the server with:

uvicorn main:app --reload

The examples below illustrate the expected request/response sequence. Run the requests in order because the items list is temporary in-memory storage and resets when the server restarts.

Task 1 – Set Up FastAPI Backend

Prompt: Install FastAPI and Uvicorn, create a FastAPI app, and provide a root endpoint.

Code explanation: FastAPI() creates the application. The GET / endpoint returns a welcome message to confirm that the server is responding.

CODE:

Visual Studio Code — main.py — Setup + root endpoint

main.py — Setup + root endpoint Python | UTF-8

```python
# main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(
    title="Lab 15 - Product API",
    description="CRUD REST API for managing products",
    version="1.0.0",
)

class Item(BaseModel):
    name: str = Field(..., example="Pen")
    price: float = Field(..., gt=0, example=20)

items: list[Item] = []

@app.get("/")
def read_root():
    """Welcome endpoint for checking the API."""
    return {"message": "Welcome to AI-powered FastAPI"}
```

OUTPUT :

Visual Studio Code — Task 1 — Root endpoint

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE

```
$ pip install fastapi uvicorn
$ uvicorn main:app --reload
INFO: Uvicorn running on http://127.0.0.1:8000
$ curl http://127.0.0.1:8000/
Expected response: {"message":"Welcome to AI-powered FastAPI"}
```

Task 2 – Create and Read Items

Prompt: Use a Python list as temporary storage and implement POST /items to add a product and GET /items to retrieve all products.

Code explanation: Item is a Pydantic model that validates a product's name and positive price. POST appends the submitted item; GET returns the current list.

CODE:

Visual Studio Code — main.py — Create and read items

main.py – Create and read items — Python | UTF-8

```python
@app.get("/items", summary="Read all items")
def get_items():
    """Return every product currently stored in memory."""
    return items

@app.post("/items", status_code=201, summary="Create an item")
def add_item(item: Item):
    """Add a product and return the created item."""
    items.append(item)
    return {"message": "Item created", "item": item}

@app.put("/items/{index}", summary="Update an item")
def update_item(index: int, item: Item):
    """Replace the product at the requested zero-based index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    items[index] = item
    return {"message": "Item updated", "item": item}
```

OUTPUT:

Visual Studio Code — Task 2 — Create and read

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE

```
$ curl http://127.0.0.1:8000/items
[]
$ curl -X POST http://127.0.0.1:8000/items -H "Content-Type: application/json" -d '{"name":"Pen","price":20}
Expected response: {"message":"Item created","item":{"name":"Pen","price":20}}
$ curl http://127.0.0.1:8000/items
[{"name":"Pen","price":20.0}]
```

Task 3 – Update an Existing Item

Prompt: Implement PUT /items/index to replace an item using its zero-based list index.

CODE:

Visual Studio Code — main.py — Create and read items

main.py – Create and read items Python | UTF-8

```python
@app.get("/items", summary="Read all items")
def get_items():
    """Return every product currently stored in memory."""
    return items

@app.post("/items", status_code=201, summary="Create an item")
def add_item(item: Item):
    """Add a product and return the created item."""
    items.append(item)
    return {"message": "Item created", "item": item}

@app.put("/items/{index}", summary="Update an item")
def update_item(index: int, item: Item):
    """Replace the product at the requested zero-based index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    items[index] = item
    return {"message": "Item updated", "item": item}
```

OUTPUT:

Visual Studio Code — Task 3 — Update item

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE

```
$ curl -X PUT http://127.0.0.1:8000/items/0 -H "Content-Type: application/json" -d '{"name":"Pencil","price"
Expected response: {"message":"Item updated","item":{"name":"Pencil","price":10}}
$ curl http://127.0.0.1:8000/items
[{"name":"Pencil","price":10.0}]
```

Task 4 – Delete an Item

Prompt: Implement DELETE /items/{index} to remove a product by index.

CODE:

Visual Studio Code — main.py — Delete + documentation

main.py – Delete + documentation Python | UTF-8

```python
@app.delete("/items/{index}", summary="Delete an item")
def delete_item(index: int):
    """Delete a product using its zero-based index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}

# Run from the terminal with:
# uvicorn main:app --reload
#
# Automatic interactive API documentation:
# http://127.0.0.1:8000/docs
# Alternative OpenAPI schema:
# http://127.0.0.1:8000/openapi.json
```

OUTPUT:

Visual Studio Code — Task 4 — Delete item

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE

```
$ curl -X DELETE http://127.0.0.1:8000/items/0
Expected response: {"message":"Item removed","item":{"name":"Pencil","price":10.0}}
$ curl http://127.0.0.1:8000/items
[]
```

Task 5 – Auto-Generated API Documentation

Prompt: Add inline comments/docstrings and make endpoint documentation available.The endpoint docstrings and summary values appear in FastAPI's Swagger UI. Open /docs to inspect methods, parameters, schemas, and try requests interactively.

CODE:

Visual Studio Code — main.py — Delete + documentation

main.py — Delete + documentation Python | UTF-8

```python
@app.delete("/items/{index}", summary="Delete an item")
def delete_item(index: int):
    """Delete a product using its zero-based index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}

# Run from the terminal with:
# uvicorn main:app --reload
#
# Automatic interactive API documentation:
# http://127.0.0.1:8000/docs
# Alternative OpenAPI schema:
# http://127.0.0.1:8000/openapi.json
```

OUTPUT:

Visual Studio Code — Task 5 — Swagger documentation

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE

```
$ uvicorn main:app --reload
Open in browser: http://127.0.0.1:8000/docs
Swagger UI lists GET /, GET /items, POST /items,
PUT /items/{index}, DELETE /items/{index}.
Each endpoint includes its summary, description,
request schema (where applicable), and response schema.
```

Conclusion

The assignment demonstrates a FastAPI backend with a root endpoint and CRUD operations for products. Request validation is handled by Pydantic, invalid indexes return HTTP 404, and interactive API documentation is available through Swagger UI at /docs. The list is temporary storage, so data is not preserved after restart.